

Trading Card Game — PC

incluant le Plan d'Assurance Qualité — version 2

Plateforme	PC (Windows)
Client	Unity 6 (6000.0.51f1) · C# 9 · .NET Standard 2.1
Serveur	.NET 8 LTS · MariaDB 10.5
Infrastructure	AWS — EC2 · S3 · Terraform
Authentification	JWT · ScryptPasswordHasher
Version document	2.0

Sommaire

I Présentation du projet

II Périmètre & acteurs

III Spécifications fonctionnelles

3.1 Interface & navigation

3.2 Gestion des decks & cartes

3.3 Matchmaking & parties

3.4 Mécanique de jeu détaillée

3.5 Gestion des utilisateurs

IV Spécifications techniques

4.1 Stack technologique

4.2 Architecture API

4.3 Sécurité & tests

V Game Design Document

5.1 Concept & piliers

5.2 Système de cartes

5.3 États des factionnaires

5.4 Archétypes & ressources

VI Direction artistique

VII Livrables & critères d'acceptation

VIII Plan Assurance Qualité (PAQ v2)

8.1 OBS — Rôles de l'équipe

8.2 Normes de développement

8.3 Gestion des tickets Jira

8.4 Méthodologie & outils

8.5 Stratégie de tests & qualité

Présentation du projet

Vortex est un jeu de cartes à collectionner (TCG) en ligne développé sous Unity, permettant à deux joueurs de s'affronter via des decks préfabriqués. Le projet naît d'un constat : les TCG existants imposent des dépenses considérables pour rester compétitif. Vortex propose un environnement **accessible économiquement**, avec une profondeur stratégique réelle et des mécaniques accessibles à tous, grâce notamment à la disponibilité de l'intégralité des cartes dès le début du jeu.

Pitch

« Des vortex menant à d'autres mondes s'ouvrent de toute part. Prenez en main une armée parmi ces univers et affrontez d'autres joueurs dans ce TCG innovant. »

Piliers d'expérience

- **Construire son deck** — composer une armée de factionnaires, sorts et équipements.
- **Utiliser son deck** — affronter un adversaire humain et déployer sa stratégie.

II Périmètre & Acteurs

Périmètre du projet

- Application de jeu développée avec **Unity** — plateforme : **PC (Windows)**
- Back-end **.NET 8** : gestion des utilisateurs, des cartes et des parties
- Authentification sécurisée par **JWT**
- Matchmaking en ligne avec synchronisation en temps réel

Acteurs du projet

Rôle	Responsabilités principales
Product Owner / Chef de projet	Supervise la production et les priorités
Game Designers	Conçoivent les mécaniques et équilibrent les cartes
Développeurs Unity	Développent le client frontend (Unity / C#)
Développeurs .NET	Développent le serveur et les APIs REST
Artistes graphiques	Produisent visuels et animations
QA	Assurent la qualité et la stabilité du jeu
Joueurs	Collectionnent, construisent des decks et jouent en ligne

III Spécifications fonctionnelles

3.1 Interface & Navigation

L'interface doit permettre à tout utilisateur d'accéder facilement aux fonctionnalités principales, depuis l'ouverture de l'application jusqu'au lancement d'une partie.

- Écran de démarrage avec animation de chargement (vortex animé + logo)
- Écran de connexion / inscription avec deux onglets dédiés
- Menu principal : **Trouver un match** · **Decks** · **Options**
- Interface responsive : compatibilité multi-résolutions, accessibilité

3.2 Gestion des decks & cartes

Chaque deck est associé à une faction, contient **30 cartes** (max. 3 exemplaires d'une même carte). Un deck incomplet peut être sauvegardé mais pas utilisé en combat.

- Liste des decks sous forme de tuiles (nom, cartes, illustration dominante)
- Création de deck : choix de faction, glisser-déposer des cartes
- Compteur en temps réel du nombre de cartes (ex : 12/30)
- Filtres : type, faction, coût en ressources · Tri : alphabétique, coût, récentes
- Sauvegarde avec confirmation, annulation, suppression avec confirmation irréversible
- Fiche détaillée au clic sur chaque carte (popup ou panneau latéral)

3.3 Matchmaking & Parties

- File d'attente avec bouton « Annuler » et lancement automatique
- Synchronisation des actions en temps réel (WebSocket / polling)
- Vérification côté serveur de toutes les actions (anti-triche)
- Écran de fin : affichage du résultat

3.4 Mécanique de jeu détaillée

Un pile ou face détermine le joueur qui commence. Chaque joueur tire **5 cartes** et reçoit **1 or** au départ. Le **premier tour** bloque la phase de bataille.

Phase	Description
Pioche	Récupération de tout l'or + 1 or supplémentaire (max 10). Pioche d'une carte. Remise à l'état initial de tous les factionnaires.
Placement 1	Jouer des cartes depuis la main : invoquer un factionnaire (arrive en repos), jouer sorts et équipements.
Bataille	Désigner les factionnaires qui attaquent. Ils passent en mode engagé après l'attaque. Déclenche la phase de défense.

Phase	Description
Placement 2	Mêmes actions qu'en Placement 1.
Défense	Bloquer les attaques adverses avec ses factionnaires disponibles (hors état repos ou engagé).

Condition de victoire — Réduire les PV du champion adverse à 0. Un abandon donne la victoire à l'adversaire par forfait.

3.5 Gestion des utilisateurs

- Inscription : pseudo (3–20 caractères, unique), e-mail valide, mot de passe ≥ 8 caractères
- Connexion : e-mail + mot de passe

IV Spécifications techniques

4.1 Stack technologique

Couche	Technologie	Version / détail
Client (frontend)	Unity + C#	Unity 6 (6000.0.51f1) · C# 9 · .NET Standard 2.1
Serveur (backend)	C# / .NET	.NET 8 LTS
Base de données	MariaDB	10.5
Authentification	JWT	ScryptPasswordHasher pour le hachage
Infrastructure	AWS (Terraform)	EC2 · S3 · MariaDB
CI/CD	GitHub Actions	Tests automatiques + merge conditionnelle

4.2 Architecture API — endpoints principaux

Endpoint	Méthode	Rôle
/api/auth/register/	POST	Inscription utilisateur
/api/auth/login/	POST	Connexion + retour JWT
/api/decks/	POST / GET	Créer / lister les decks
/api/decks/{id}/	GET / PUT / DELETE	Lire, modifier, supprimer un deck

4.3 Sécurité & tests

- Hachage des mots de passe : **ScryptPasswordHasher** côté serveur
- Authentification **JWT obligatoire** sur toutes les routes protégées
- Validation de toutes les actions de jeu côté serveur (anti-triche)
- Chiffrement des données personnelles, conformité **RGPD**
- Couverture de code cible : **≥ 75 %**



Game Design Document (résumé)

5.1 Concept & piliers d'expérience

Vortex repose sur deux piliers : **construire son deck** (composer une armée de factionnaires, sorts et équipements) et **utiliser son deck** (affronter un adversaire humain et déployer sa stratégie). La stratégie est la composante numéro un : chaque archétype doit disposer de réponses à toutes les situations.

5.2 Système de cartes

Les cartes affichent : **nom**, **splashart**, **statistiques dynamiques** sur le terrain, et **effets en texte formaté** (nom en gras + description). L'intégralité des cartes est disponible dès le début du jeu.

5.3 États des factionnaires

État	Type	Effet
Repos	Négatif	Empêche toute action. S'applique à l'invocation.
Engagé	Négatif	Empêche toute action. S'applique après l'attaque / défense.

5.4 Archétypes

Chaque archétype représente un univers unique avec ses propres cartes, sa **monnaie secondaire** (récupérée via des actions propres à la faction) et son **champion** (compétence active, 1 fois/tour).

– Archétypes en développement : **Cactus**, **Berzerker** (d'autres à définir)

5.5 Ressources

Ressource	Description
Or	Monnaie principale. Sert à jouer les cartes. Régénère entièrement + 1 à chaque début de tour (max 10).
Monnaie secondaire	Propre à chaque faction. Récupérée via des actions spécifiques à l'archétype.

VI Direction artistique & son

L'univers de Vortex est **décalé et loufoque** (ex : vampires contre cactus). Chaque monde possède une identité visuelle forte, cohérente avec son archétype.

- Cartes affichant : nom, splashart, statistiques dynamiques sur le terrain
- Effets en texte formaté (nom en gras + description)
- Direction artistique complète détaillée dans le **Moodboard dédié**
- Animation de chargement : vortex animé + logo

VII Livrables & Critères d'acceptation

Livrables attendus

- Application Unity jouable sur PC
- Back-end .NET 8 déployé et accessible
- Code source versionné sur GitHub
- Base de données MariaDB configurée
- Documentation technique (API, architecture)
- Présentation / démonstration du projet

Definition of Done — Critères d'acceptation

Fonctionnalité	Critère d'acceptation
Inscription / Connexion	Créer un compte et se connecter sans erreur
Deck builder	Créer, modifier et supprimer un deck de 30 cartes
Matchmaking	Trouver et lancer une partie à 2 joueurs en moins de 30 s
Partie complète	Jouer jusqu'à la victoire sans bug bloquant
Synchronisation	Les actions d'un joueur sont visibles par l'adversaire en temps réel
Sécurité	Aucune action illégale n'est acceptée côté serveur
Performance	FPS stable sur la configuration cible

VIII

Plan Assurance Qualité — PAQ v2

8.1 OBS — Rôles de l'équipe

Product Owner — Alex

- Définit la vision produit et les objectifs à court/moyen terme
- Priorise les fonctionnalités et gère le backlog
- Communique avec l'équipe pour s'assurer de la bonne compréhension des besoins
- Vérifie que le produit répond aux attentes des utilisateurs finaux

DevOps — Maxime

- Met en place les environnements de développement, test et production
- Automatise les processus de build, test et déploiement (CI/CD)
- Assure la sécurité, la stabilité et la scalabilité de l'infrastructure
- Surveille les performances du système et résout les incidents
- Gère les conteneurs, images Docker, pipelines, etc.

Lead Developer Back — Mikaël

- Structure l'architecture du back-end
- Met en place les API nécessaires au bon fonctionnement de l'application
- Supervise les tests et la qualité du code serveur
- Guide les développeurs back dans leurs choix techniques
- Documente les fonctionnalités du back-end

Game Designer — Alex

- Conçoit les mécaniques de jeu (gameplay, logique)
- Propose des idées d'amélioration pour l'expérience de jeu

Lead Developer Front & UI/UX Designer — Clara

- Supervise l'intégration visuelle et interactive côté front-end
- Assure la cohérence UI entre les écrans et les interactions
- Accompagne les devs front sur les décisions techniques
- Crée des maquettes

Développeurs — Clara, Mikaël

- Implémentent les fonctionnalités front ou back selon les besoins
- Écrivent des tests pour valider leur code
- Participent aux revues de code et aux sprints
- Communiquent avec les leads pour bien comprendre les tâches
- Corrigent les bugs et améliorent le code existant

QA TEST — Alex

- Teste les fonctionnalités

8.2 Normes de développement

Branches Git

Branche	Description
main	Production. Merge à la fin de chaque sprint lors des rétrospectives.
dev	Branche de développement principale.
staging	Branche dédiée aux tests de tickets.
prenom/vor-numero-nom	Branche générée automatiquement par Jira à partir du ticket attribué.

Règles de merge — branche dev

- Les développeurs ne peuvent pas merger directement sur **dev**
- Une **Pull Request (PR)** est obligatoire pour merger d'une branche feat/ vers dev
- Une review d'un autre développeur est requise avant validation du merge
- Les PR sont mergées par le **DevOps**

Règles de merge — branche main

- Les développeurs ne peuvent pas merger sur **main**
- Seul le **PO** peut merger, uniquement depuis la branche staging

Conventions de commit

Format : **[TYPE] Description claire du commit**

[FEAT]	Ajout d'une nouvelle fonctionnalité
[FIX]	Correction de bug
[EVO]	Amélioration d'une fonctionnalité existante
[STYLE]	Modifications visuelles
[CLEAN]	Nettoyage, indentation, suppression de code mort

Conventions de nommage

Élément	Convention
Fichiers, Classes, Fonctions, Variables	PascalCase

8.3 Gestion des tickets — Jira

Statut	Description
Backlog	Regroupe tous les tickets. Sélectionnés ou mis à jour lors des sprint plannings.
To Do	Tickets sélectionnés pour le sprint en cours.
In Progress	Tickets en cours de développement.
Testing	Tickets en attente de tests.
Review	Tickets à relire par le Product Owner.
Done	Tickets finalisés et validés.

8.4 Méthodologie du projet

Réunion d'équipe hebdomadaire

Chaque jeudi à 9h30 — Bilan d'avancement et blocages. Compte-rendu publié sur Notion.

Outils utilisés

Outil	Usage
GitHub	Versioning du code source
Jira	Gestion des tâches et tickets
Discord	Communication d'équipe
Notion	Documentation centralisée et livret d'accueil

8.5 Stratégie de tests & qualité

Des **tests unitaires**, **d'intégration** et **fonctionnels** seront progressivement mis en place. Chaque fonctionnalité devra être couverte par un **ticket de test dédié**, bloquant pour la mise en production s'il n'est pas validé.

Types de tests

Type	Périmètre
Test unitaire	Logique back-end (fonctions, services, validations)
Test d'intégration	Communication entre composants / API
Test fonctionnel	Scénarios utilisateur end-to-end
Test de sécurité	Création de compte · Connexion · Authenticité de l'utilisateur connecté

Structure d'un ticket de test

- **Fonctionnalité concernée**
- **Tests associés** (unitaires et/ou d'intégration)

Couverture & qualité

- Couverture unitaire de **100 %** exigée avant toute mise en production
- Un **linter** mis en place pour détecter erreurs et alertes de code

Rapport de test

Avant chaque déploiement, un **rapport de test** est généré comme valeur étalon pour les métriques suivantes :

- Pourcentage de couverture de tests
- Nombre d'erreurs détectées par le linter

Onboarding

Un **livret d'accueil** est proposé aux nouveaux arrivants. L'ensemble de la documentation est centralisé sur **Notion**.